

Лабораторная работа №21. Создание приложения TODO

Цель лабораторной работы: Создать простой TODO-лист

Шаг 1. Подготовка проекта

1. Откройте **Visual Studio Code**
2. Откройте встроенный терминал
3. Перейдите в каталог с документами
4. Создайте директорию для лабораторной работы:
Lab21_DOM_Events_FIO (где FIO — ваша фамилия)
5. Перейдите в созданную директорию
6. Создайте структуру проекта:
 - **README.md**
 - **index.html**
 - **style.css**
 - **main.js**
7. Откройте созданную папку в **Visual Studio Code**

Инициализация Git-репозитория

1. Инициализируйте Git-репозиторий:

git init

2. Добавьте файлы в индекс:

git add .

3. Создайте первый коммит:

git commit -m "Initial commit"

Создание удалённого репозитория

1. Создайте пустой **public-репозиторий** на GitHub с именем:

Lab21_ToDo

2. Привяжите локальный репозиторий к удалённому:

git branch -M main

git remote add origin <URL_репозитория>

git push -u origin main

3. Сделайте **скриншот терминала** с выполненными командами:

- **commit**
- **remote add**

- **push**

4. Создайте папку **img** и сохраните в ней изображение удачного пуша под именем:

gitPushLab21_DOM_Events_FIO.png

5. Добавьте изменения и отправьте их:

git add .

git commit -m "Added push screenshot and project update"

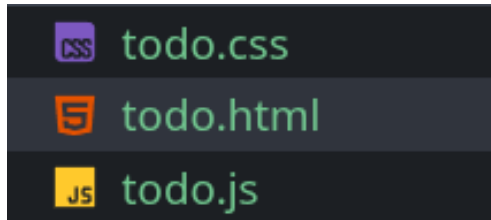
git push

Шаг 2. Введение в TODO-лист

TODO-лист (список задач) — это классическое приложение для демонстрации работы с DOM. Он объединяет всё изученное:

- Поиск и создание элементов
- Обработку событий
- Работу с формами
- Динамическое обновление интерфейса

Создайте **три** новых файла в проекте:



- **todo.html**
- **todo.css**
- **todo.js**



Открывайте в браузере нужный файл в зависимости от шага

Добавьте основную разметку в файл **todo.html**:

```
<!DOCTYPE html>
<html lang="ru">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Простой TODO-лист</title>
    <link rel="stylesheet" href="todo.css" />
  </head>
  <body> ...
</body>
</html>
```

Внутри **body** добавим **div** с классом и подключим скрипт:

```

<body>
  <div class="container">...
</div>
+
<script src="todo.js"></script>
</body>

```

Далее в наш основной контейнер вставим заголовок, блок для добавления новых задач, список задач (будет заполняться через JavaScript), блок с информацией и счётчиками:

```

<body>
  <div class="container">
    <h1>Мой TODO-лист</h1>
    <div class="add-task">
      <input type="text" id="taskInput" placeholder="Новая задача..." />
      <button id="addBtn">Добавить</button>
    </div>
    <ul id="taskList">
      <!-- Задачи будут добавляться сюда -->
    </ul>
    <div class="info">
      <p id="taskCounter">Всего задач: 0</p>
    </div>
  </div>
  <script src="todo.js"></script>
</body>

```

Откроем в браузере **todo.html** и проверим нашу страницу:

Мой TODO-лист

Новая задача...

Всего задач: 0

Далее перейдём к написанию стилей **todo.css**

1. Сброс стандартных отступов и применение border-box:

```

1  * {
2    margin: 0;
3    padding: 0;
4    box-sizing: border-box;
5  }

```

2. Стили всей страницы:

```
7  body {
8    font-family: Arial, sans-serif;
9    background-color: #f5f5f5;
10   padding: 20px;
11   line-height: 1.6;
12 }
```

3. Основной контейнер приложения:

```
14 .container {
15   max-width: 600px;
16   margin: 0 auto;
17   background-color: white;
18   padding: 30px;
19   border-radius: 10px;
20   box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
21 }
```

4. Заголовок:

```
23 h1 {
24   text-align: center;
25   color: #333;
26   margin-bottom: 20px;
27 }
```

5. Блок добавления задачи:

```
29 .add-task {
30   display: flex;
31   gap: 10px;
32   margin-bottom: 20px;
33 }
```

6. #taskInput (занимаем всё доступное пространство):

```
35 #taskInput {
36   flex: 1;
37   padding: 10px 15px;
38   border: 2px solid #ddd;
39   border-radius: 5px;
40   font-size: 16px;
41 }
```

7. #taskInput:focus (добавляем зелёную рамку при фокусе):

```
43 #taskInput:focus {
44   outline: none;
45   border-color: #4caf50;
46 }
```

8. #addBtn

```
48 #addBtn {
49   padding: 10px 20px;
50   background-color: #4caf50;
51   color: white;
52   border: none;
53   border-radius: 5px;
54   cursor: pointer;
55   font-size: 16px;
56   transition: background-color 0.3s;
57 }
```

9. #addBtn:hover (делаем темнее при наведении):

```
59 #addBtn:hover {
60   background-color: #45a049;
61 }
```

10. Список задач (убираем маркеры списка):

```
63 #taskList {
64   list-style-type: none;
65 }
```

11. Элемент задачи:

```
67 .task-item {
68   display: flex;
69   align-items: center;
70   padding: 12px 15px;
71   margin-bottom: 8px;
72   background-color: #f9f9f9;
73   border-left: 4px solid #4caf50;
74   border-radius: 5px;
75   transition: all 0.3s;
76 }
```

12. `.task-item:hover` (добавляет подсветку при наведении):

```
78  .task-item:hover {
79  |  background-color: #f0f0f0;
80  }
```

13. Текст задачи (занимает всё доступное пространство):

```
82  .task-text {
83  |  flex: 1;
84  |  font-size: 16px;
85  }
```

14. `.task-text.completed` (добавляем зачёркивание выполненной задачи и цвет выполненной):

```
87  .task-text.completed {
88  |  text-decoration: line-through;
89  |  color: #888;
90  }
```

15. Контейнер для кнопок задачи:

```
92  .task-buttons {
93  |  display: flex;
94  |  gap: 8px;
95  }
```

16. Общие стили кнопок:

```
97  .complete-btn,
98  .delete-btn {
99  |  padding: 5px 10px;
100 |  border: none;
101 |  border-radius: 3px;
102 |  cursor: pointer;
103 |  font-size: 14px;
104 }
```

17. Кнопка "Выполнено" (меняем цвета):

```
106 .complete-btn {
107 |  background-color: #2196f3;
108 |  color: white;
109 }
```

18. `.complete-btn:hover` (делаем темнее при наведении):

```
111 .complete-btn:hover {  
112   background-color: #1976d2;  
113 }
```

19. Кнопка "Удалить" (меняем цвета):

```
115 .delete-btn {  
116   background-color: #f44336;  
117   color: white;  
118 }
```

20. `.delete-btn:hover` (делаем темнее при наведении):

```
120 .delete-btn:hover {  
121   background-color: #d32f2f;  
122 }
```

21. Информационный блок (добавляем отступы, разделительную линию и выравнивание по центру):

```
124 .info {  
125   margin-top: 20px;  
126   padding-top: 15px;  
127   border-top: 1px solid #eee;  
128   text-align: center;  
129 }
```

22. Меняем цвет и начертание счётчика:

```
131 #taskCounter {  
132   color: #666;  
133   font-weight: bold;  
134 }
```

Теперь перейдём к написанию `todo.js`

1. В начале мы хотим получить элементы DOM. Поэтому находим HTML-элементы по их id и сохраняем в переменные (поле ввода, кнопку «Добавить», список задач (ul), счётчик задач):

```

1 const taskInput = document.getElementById("taskInput");
2 const addBtn = document.getElementById("addBtn");
3 const taskList = document.getElementById("taskList");
4 const taskCounter = document.getElementById("taskCounter");

```

Далее создадим счётчик задач, который будет нашей глобальной переменной:

```

5 let totalTasks = 0;

```

2. Напишем функцию **updateCounter()**. Она будет обновлять текст счётчика задач на странице. Как меняется страница: Текст "Всего задач: X" меняется на актуальное число:

```

7 // Функция для обновления счетчика
8 function updateCounter() {
9   taskCounter.textContent = `Всего задач: ${totalTasks}`;
10 }

```

3. Напишем функцию **createTaskItem()**. Она будет создавать HTML-элемент для одной задачи (**все конструкции пишем внутри функции**):

```

// Функция для создания новой задачи
function createTaskItem(text) { ...
}

```

3.1. Создаём элементы "с нуля". элемент списка, текст задачи, контейнер для кнопок, кнопка "Выполнено", кнопка "Удалить":

```

12 // Функция для создания новой задачи
13 function createTaskItem(text) {
14   // Создаем элементы
15   const taskItem = document.createElement("li");
16   const taskText = document.createElement("span");
17   const buttonsDiv = document.createElement("div");
18   const completeBtn = document.createElement("button");
19   const deleteBtn = document.createElement("button");

```

3.2. Далее настраиваем CSS-классы для стилизации:

```

21 // Настраиваем классы
22 taskItem.className = "task-item";
23 taskText.className = "task-text";
24 buttonsDiv.className = "task-buttons";
25 completeBtn.className = "complete-btn";
26 deleteBtn.className = "delete-btn";

```


3.3. Заполняем элементы текстом:

```
28 // Устанавливаем текст
29 taskText.textContent = text;
30 completeBtn.textContent = "Выполнено";
31 deleteBtn.textContent = "Удалить";
```

3.4. Собираем структуру дерева DOM:

```
33 // Собираем структуру
34 buttonsDiv.appendChild(completeBtn);
35 buttonsDiv.appendChild(deleteBtn);
36 taskItem.appendChild(taskText);
37 taskItem.appendChild(buttonsDiv);
```

3.5. Добавляем обработчики событий:

```
39 // Добавляем обработчики событий
40 completeBtn.addEventListener("click", function () {
41     taskText.classList.toggle("completed");
42     completeBtn.textContent = taskText.classList.contains("completed")
43         ? "Возобновить"
44         : "Выполнено";
45 });
```

3.6. Обработчик для кнопки "Удалить":

```
47 // Обработчик для кнопки "Удалить"
48 deleteBtn.addEventListener("click", function () {
49     taskItem.remove();
50     totalTasks--;
51     updateCounter();
52 });
```

3.7. Возвращаем готовый элемент задачи:

```
54 return taskItem;
```

4. Напишем функцию **addNewTask()**. Она будет добавлять новую задачу в список:

```
// Функция добавления новой задачи
function addNewTask() { ...
}
```

(все конструкции пишем внутри функции)

4.1. Получаем текст из поля ввода, обрезаем пробелы:

```
58 function addNewTask() {  
59   const text = taskInput.value.trim();
```

4.2. Проверяем, не пустая ли задача:

```
61   // Проверяем, не пустая ли задача  
62   if (text === "") {  
63     alert("Пожалуйста, введите текст задачи");  
64     return;  
65   }
```

4.3. Создаём элемент задачи (вызываем функцию выше) и добавляем её в конец списка:

```
67   // Создаем и добавляем задачу  
68   const newTask = createTaskItem(text);  
69   taskList.appendChild(newTask);
```

4.4. Обновляем счётчик:

```
71   // Обновляем счетчик  
72   totalTasks++;  
73   updateCounter();
```

4.5. Очищаем поле ввода и ставим курсор обратно в поле ввода:

```
75   // Очищаем поле ввода  
76   taskInput.value = "";  
77   taskInput.focus();
```

5. Настроим обработчик событий. Когда пользователь нажимает кнопку "Добавить" будет появляться новая задача:

```
80   // Обработчики событий  
81   addBtn.addEventListener("click", addNewTask);
```

6. Когда пользователь нажимает Enter в поле ввода будем вызывать ту же функцию:

```
83   // Добавляем возможность нажатия Enter  
84   taskInput.addEventListener("keypress", function (event) {  
85     if (event.key === "Enter") {  
86       addNewTask();  
87     }  
88   });
```

7. В качестве примера, при инициализации нашей страницы при загрузке, создадим три простые задачи:

```
document.addEventListener("DOMContentLoaded", function () {
  const sampleTasks = [
    "Изучить JavaScript",
    "Создать TODO-лист",
    "Поработать над проектом",
  ];
  sampleTasks.forEach(function (task) {
    const newTask = createTaskItem(task);
    taskList.appendChild(newTask);
    totalTasks++;
  });
  updateCounter();
});
```

Практическое задание

1. Сделайте скриншот страницы с результатом работы в папку **img** с именем:

todoBasicLab21_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "basic TODO list implementation"

git push

Самостоятельное задание №1

Цель задания: Создать подробное и структурированное описание лабораторной работы №21 в файле README.md, используя Markdown.

Что должно быть в README.md

1. Заголовок

2. Основная информация

****ФИО:** Иванов Иван Иванович**

****Группа:** ИСП-231**

****Дата:** 18.02.2026**

3. Краткое описание работы

Описание (что изучили)

4. Структура проекта

Структура проекта

Самостоятельное задание № 2. Секундомер / таймер

Создайте самостоятельно приложение секундомер / таймер.

Функционал:

1. Отображение времени (00:00:00)
2. Кнопки «Старт», «Стоп», «Сброс»
3. Обновление DOM каждую секунду
4. Возможность запускать и останавливать несколько раз

Пример коммитов:

1. *Initial project: timer layout*
2. *Added CSS for timer*
3. *Implemented start/stop functionality*
4. *Added reset button*

Скриншот: **timer_FIO.png**

Финальный коммит (после всех шагов):

git add .

git commit -m "Completed Lab 21"

git push